

```

import numpy as np
from sklearn.datasets import load_iris

def euclidean_distance(p, q):
    return np.sqrt(np.sum((p - q) ** 2))

def manhattan_distance(p, q):
    return np.sum(np.abs(p - q))

def minkowski_distance(p, q, r):
    return np.sum(np.abs(p - q) ** r) ** (1 / r)

def chebyshev_distance(p, q):
    return np.max(np.abs(p - q))

def squared_euclidean_distance(p, q):
    return np.sum((p - q) ** 2)

def cosine_distance(p, q):
    dot_product = np.dot(p, q)
    magnitude_p = np.linalg.norm(p)
    magnitude_q = np.linalg.norm(q)

    if magnitude_p == 0 or magnitude_q == 0:
        return 1.0

    cosine_similarity = dot_product / (magnitude_p * magnitude_q)
    return 1 - cosine_similarity

iris = load_iris()
X = iris.data
feature_names = iris.feature_names

print("=" * 70)
print("      DISTANCE MEASURES IN N-DIMENSIONAL FEATURE SPACE")
print("=" * 70)

```

```

print("\nDataset: Iris Dataset")
print("Number of samples :", X.shape[0])
print("Number of features:", X.shape[1])

print("\nFeatures:")
for i, name in enumerate(feature_names):
    print(f"{i + 1}. {name}")

print("\n" + "=" * 70)
print("FIRST 10 SAMPLE POINTS")
print("=" * 70)

print("\nSample\tSepal Length\tSepal Width\tPetal Length\tPetal Width")

for i in range(10):
    print(
        f"{i}\t"
        f"{X[i][0]:.2f}\t\t"
        f"{X[i][1]:.2f}\t\t"
        f"{X[i][2]:.2f}\t\t"
        f"{X[i][3]:.2f}"
    )

print("\n" + "=" * 70)
print("SELECT TWO SAMPLE POINTS")
print("=" * 70)

point1_index = int(input("\nEnter index of first point (0-149): "))
point2_index = int(input("Enter index of second point (0-149): "))

if point1_index < 0 or point1_index >= len(X):
    print("Invalid first point index.")
    exit()

if point2_index < 0 or point2_index >= len(X):

```

```
print("Invalid second point index.")
exit()
```

```
P = X[point1_index]
Q = X[point2_index]
```

```
print("\nPoint P:")
print(P)
```

```
print("\nPoint Q:")
print(Q)
```

```
print("\nDifference P - Q:")
print(P - Q)
```

```
while True:
    print("\n" + "=" * 70)
    print("DISTANCE MEASURES")
    print("=" * 70)
```

```
print("1. Euclidean Distance")
print("2. Manhattan Distance")
print("3. Minkowski Distance")
print("4. Chebyshev Distance")
print("5. Squared Euclidean Distance")
print("6. Cosine Distance")
print("7. Calculate ALL distances")
print("8. Exit")
```

```
choice = int(input("\nEnter your choice: "))
```

```
if choice == 1:
    d = euclidean_distance(P, Q)
    print("\nEuclidean Distance =", d)
```

```
elif choice == 2:
```

```

d = manhattan_distance(P, Q)
print("\nManhattan Distance =", d)

elif choice == 3:
    r = float(input("\nEnter Minkowski parameter r: "))

    if r <= 0:
        print("r must be greater than 0.")
    else:
        d = minkowski_distance(P, Q, r)
        print("\nMinkowski Distance =", d)

elif choice == 4:
    d = chebyshev_distance(P, Q)
    print("\nChebyshev Distance =", d)

elif choice == 5:
    d = squared_euclidean_distance(P, Q)
    print("\nSquared Euclidean Distance =", d)

elif choice == 6:
    d = cosine_distance(P, Q)
    print("\nCosine Distance =", d)

elif choice == 7:
    print("\n" + "=" * 70)
    print("DISTANCE RESULTS")
    print("=" * 70)

    print(
        "Euclidean Distance          :",
        euclidean_distance(P, Q)
    )

    print(
        "Manhattan Distance              :",

```

```

        manhattan_distance(P, Q)
    )

    print(
        "Chebyshev Distance      :",
        chebyshev_distance(P, Q)
    )

    print(
        "Squared Euclidean Distance:",
        squared_euclidean_distance(P, Q)
    )

    print(
        "Cosine Distance              :",
        cosine_distance(P, Q)
    )

    r = 3

    print(
        f"Minkowski Distance (r={r}):",
        minkowski_distance(P, Q, r)
    )

elif choice == 8:
    print("\nProgram terminated.")
    break

else:
    print("\nInvalid choice. Please select 1-8.")

```